

PDE4444

Machine Learning for Engineers

Week 8



Machine Learning Workflow

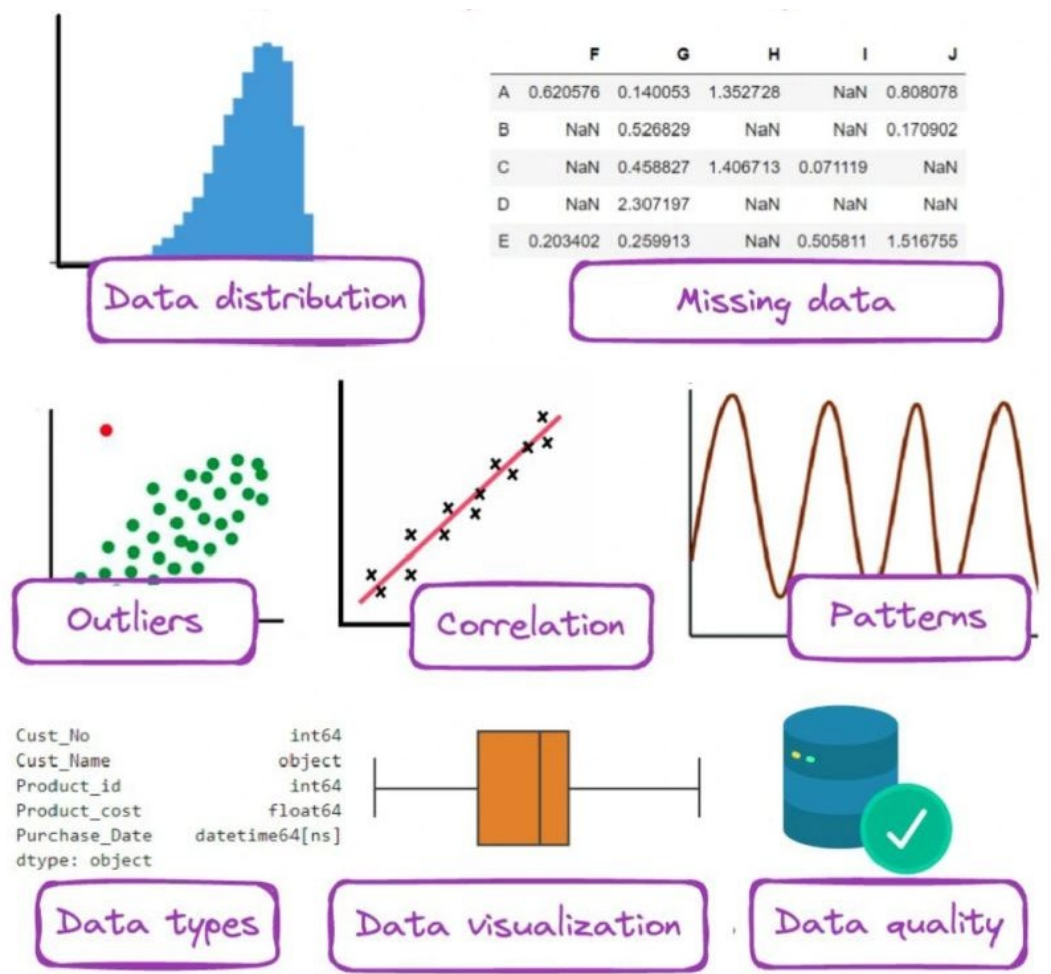
The general structure of a machine learning study/project stays relatively constant:

1. Data cleaning and formatting ✓
2. Exploratory data analysis
3. Feature engineering and selection
4. Establish a baseline and compare several machine learning models on a performance metric
5. Perform hyperparameter tuning on the best model to optimize it for the problem
6. Evaluate the best model on the testing set
7. Interpret the model results to the extent possible
8. Draw conclusions and write a well-documented report



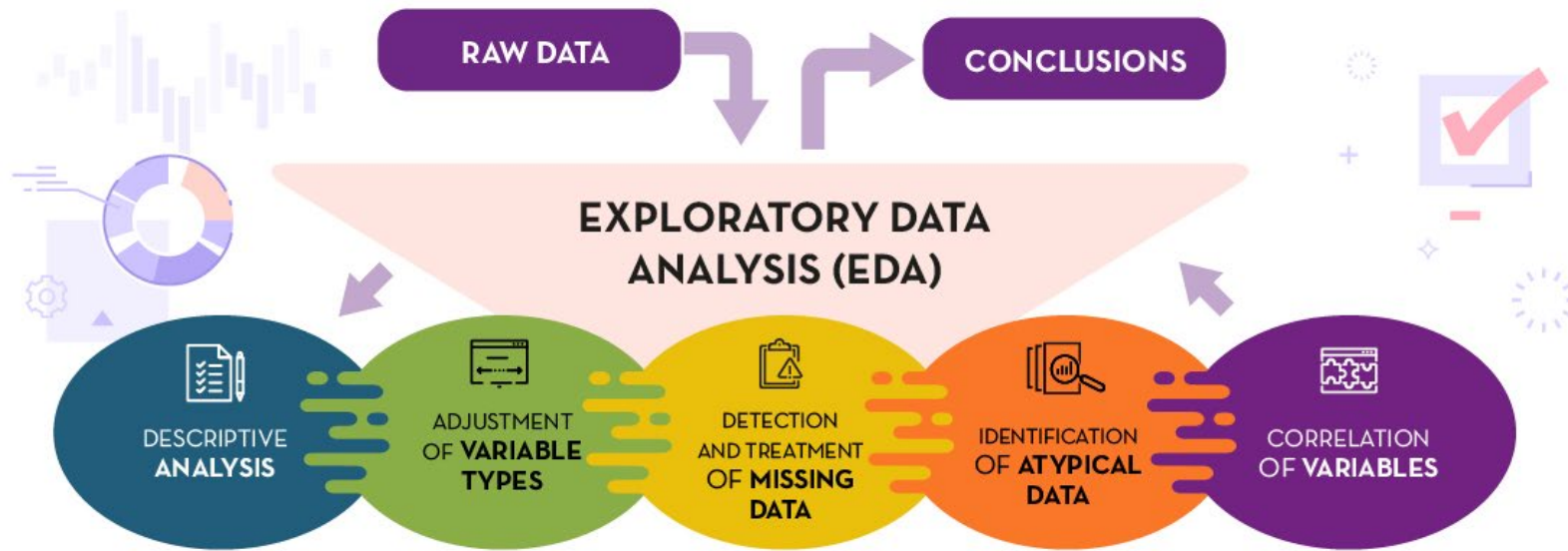
Understanding Exploratory Data Analysis (EDA)

EDA helps in discovering insights before model training.



Introduction to Exploratory Data Analysis (EDA)

- **EDA** is the process of analysing datasets to summarise key characteristics
- Helps in understanding patterns, detecting outliers, and making data-driven decisions.
- Crucial step before applying Machine Learning models.



Tools and Techniques for EDA

- 1. Distribution Analysis**- a fundamental technique used in EDA. It helps us understand how the data points in our dataset are distributed across its range. It also helps in identifying any outliers and is often represented using histograms, density plots, and box plots.
- 2. Correlation Analysis** - helps to identify the relationships between various variables in our dataset. Understanding these dependencies is an important concept in machine learning, as it allows the selection of relevant predictors for your model and avoids multicollinearity.
- 3. Outlier Detection** - Effective outlier detection is a crucial phase in EDA. Using tools such as box plots or scatter plots can help recognize anomalies in your data. Predictive modelling greatly benefits from a clean dataset, free of outliers. Specialised algorithms for outlier detection can be incorporated into your analysis process for further refinement and optimization.



Tools and Techniques for EDA

4. Multivariate Analysis - is a crucial aspect of EDA. It utilizes statistical methods to help decipher patterns or relationships between multiple data variables simultaneously. This technique offers a profound insight into the dataset, helping knowledge workers make informed decisions.

5. Time Series Visualization - Decoding patterns and seasonal trends through time series visualization is a powerful component. With interactive tools, knowledge workers can visually examine fluctuations over a period, facilitating the recognition of anomalies and forecasting.

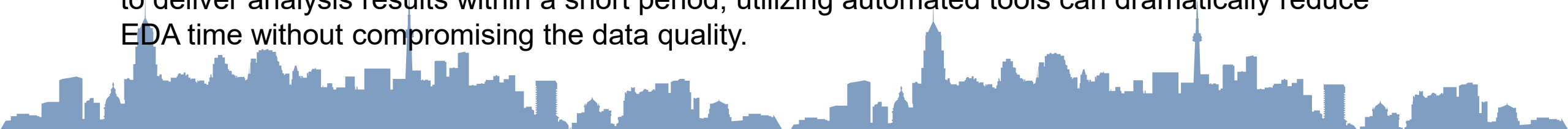
6. Dimensionality Reduction - To handle complex datasets with a large number of features, efficient dimensionality reduction techniques are critical. Techniques such as PCA and t-SNE offer effective ways to reduce data dimensionality, speeding up the learning process without losing important information.

7. Interactive Visualizations- Exploring data through Interactive data visualization tools can significantly help gain a better understanding. Tools such as MarkovML provide a graphical interface where knowledge workers can manipulate data characteristics and observe changes. These tools are instrumental in predictive modelling, where instant visual feedback can help identify patterns and trends effectively.



Challenges in EDA

- 1. Insufficient Data** - When you don't have a considerable amount of data for analysis, you are dealing with the "insufficient data" problem. It can jeopardize the ML modeling process by providing inaccurate results and misleading insights. Using data augmentation techniques, such as bootstrapping or data imputations, or even using synthetic data can be handy here.
- 2. Noisy data** - refers to the irrelevant or misleading information present within the dataset. It has a significant impact on the performance of ML models. Effective data cleaning and imputation methods should be applied to ensure data quality. Interactive data visualization tools are often helpful in identifying and removing noise from the data.
- 3. High-dimensional data** - include too many features or variables, making data exploration and visualization complex. Dimensionality reduction in data science is a method used to simplify such datasets without losing much information. For instance, statistical methods such as Principal Component Analysis (PCA) are commonly used for this kind of issue.
- 4. Imbalanced Datasets** - are those in which the outcome classes are not represented equally. Such imbalance can lead to misleading insights in ML modeling. Using handling imbalanced datasets techniques such as resampling or applying cost-sensitive learning can help deal with this problem efficiently.
- 5. Time Constraints** - can be a significant constraint in exploratory data analysis. When you need to deliver analysis results within a short period, utilizing automated tools can dramatically reduce EDA time without compromising the data quality.



Feature Engineering and Selection

- Feature engineering is the process of transforming raw data into meaningful inputs that improve model performance.

→ EDA helps us **understand** the data.

Feature engineering helps us **improve** the data.



Feature Engineering and Selection

- **Feature Engineering:** The process of taking raw data and extracting or creating new features that allow a ML model to learn a mapping between these features and the target. This might mean taking transformations of variables, such as we did with the log and square root, or one-hot encoding categorical variables so they can be used in a model. Generally, we can think of feature engineering as **adding** additional features derived from the raw data.
- **Feature Selection:** The process of choosing the most relevant features in your data. In feature selection, we remove features that do not help our model learn the relationship between features and the target. This can help the model generalize better to new data and results in a more interpretable model. Generally, we can think of feature selection as **subtracting** features so we are left with only those that are most important.



Additional Feature Selection

There are plenty of more methods for [feature selection](#).

Some popular methods include:

- Principal components analysis (PCA) which transforms the features into a reduced number of dimensions that preserve the greatest variance
- Independent components analysis (ICA) which aims to find the independent sources in a set of features.

These methods are very helpful for dealing with high-dimensional data. However, while these methods are effective at reducing the number of features, they create new features that have no physical meaning and hence make interpreting a model nearly impossible.



Problem Definition

- We are going to use an example that takes us through a complete machine learning problem using a real-world dataset.

Use the provided building energy data to develop a model that can predict a building's Energy Star score, and then interpret the results to find the variables that are most predictive of the score.



Problem Definition

- This is a supervised, regression machine learning task: given a set of data with targets (in this case the energy score) included, we want to train a model that can learn to map the features (also known as the explanatory variables) to the target.
 - Supervised problem: we are given both the features and the target
 - Regression problem: the target is a continuous variable, in this case ranging from 0-100
- During training, we want the model to learn the relationship between the features and the score, so we give it both the features and the answer. Then, to test how well the model has learned, we evaluate it on a testing set where it has never seen the answers!



Understand your Dataset

Download the Energy_Data.csv file.

Download the Benchmarking_data_definitions.pdf document.

(Take some time to study both files)

We hypothesize that the dataset contains independent variables that might be used to infer interesting facts about buildings in New York. Of primary interest to us is the Energy Star score, since we believe this score is used as an aggregate measure of a building's energy performance.

Questions we are trying to answer:

- Does this dataset contain a set of independent variables that correlate to the energy star rating of the building?
- For buildings without energy star scores, can you infer what their score might be?



Imports

- We will use the standard data science libraries: `numpy`, `panda`.
- We also use `matplotlib` and `seaborn` for visualization.

```
# Pandas and numpy for data manipulation
import pandas as pd
import numpy as np
# Display up to 60 columns of a dataframe
pd.set_option('display.max_columns', 60)
# Matplotlib visualization
import matplotlib.pyplot as plt
# Seaborn for visualization
import seaborn as sns
```



Data Cleaning and Formatting

Load in the Data and Examine

- We will be loading our data into a pandas dataframe.
- Pandas has many methods to help make the data science/machine learning pipeline as smooth as possible.

```
# Read in data into a dataframe  
data = pd.read_csv('Energy_Data.csv')
```

```
# Display top of dataframe  
data.head()
```



Data Cleaning and Formatting

Data Types and Missing Values

- The `dataframe.info` method is a quick way to assess the data by displaying the data types of each column and the number of non-missing values. Already from looking at the dataframe, there might be a problem because the missing values are coded as "**Not Available**" rather than as `np.nan` (not a number). This means the columns with numbers will not be represented as numeric because pandas converts columns with any strings values into columns of all strings.

```
# See the column data types and non-missing values  
data.info()
```

There are a number of columns with numbers that have been recorded as `object` datatypes. These will have to be converted to `float` datatype before we can do any numerical analysis.



Data Cleaning and Formatting

Convert Data to Correct Types

- We convert the columns with numbers into numeric data types by replacing the **"Not Available"** entries with `np.nan` which can be interpreted as floats. Then we will convert the columns that contain numeric values (such as square feet or energy usage) into numeric datatypes.

```
# Replace all occurrences of Not Available with numpy not a number
data = data.replace({'Not Available': np.nan})
# Iterate through the columns
for col in list(data.columns):
    # Select columns that should be numeric
    if ('ft²' in col or 'kBtu' in col or 'Metric Tons CO2e' in col or 'kWh' in
        col or 'therms' in col or 'gal' in col or 'Score' in col):
        # Convert the data type to float
        data[col] = data[col].astype(float)
# Statistics for each column
data.describe()
```



Data Cleaning and Formatting

Understanding the Percentiles (Quartiles)

1.25% (First Quartile, Q1)

1. This is the **first quartile (Q1)**, also known as the **lower quartile**.
2. It represents the **25th percentile**, meaning **25% of the data points are below this value**, and 75% are above it.
3. It is the **median of the lower half** of the dataset.

2.50% (Second Quartile, Q2 or Median)

1. This is the **median (Q2)** of the data.
2. It represents the **50th percentile**, meaning **half of the data points are below this value**, and half are above it.
3. It is the **middle value** of the dataset when sorted.

3.75% (Third Quartile, Q3)

1. This is the **third quartile (Q3)**, also known as the **upper quartile**.
2. It represents the **75th percentile**, meaning **75% of the data points are below this value**, and only 25% are above it.
3. It is the **median of the upper half** of the dataset.



Data Cleaning and Formatting

How Percentiles Help in Data Analysis

- **Identifying Skewness:** If the **median (50%)** is closer to the **25% or 75%**, the data might be skewed.
- **Detecting Outliers:** If **min/max values** are far from Q1/Q3, there may be outliers.
- **Understanding Distribution:** If the distance between **Q1 & Q2** is different from **Q2 & Q3**, the data is not uniformly distributed.



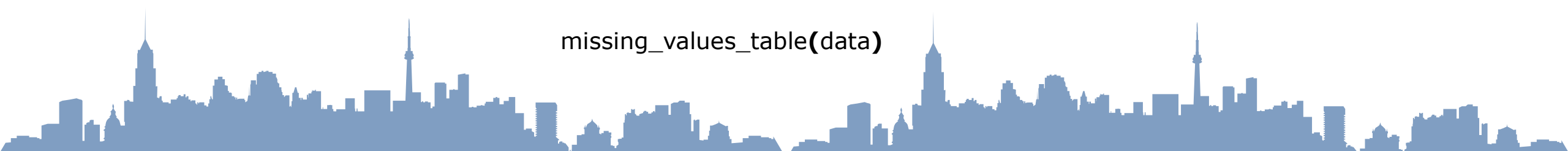
Data Cleaning and Formatting

Missing Values

- Now that we have the correct column datatypes, we can start analysis by looking at the percentage of missing values in each column.
- Following is a function that calculates the number of missing values and the percentage of the total values that are missing for each column.

```
# Function to calculate missing values by column
def missing_values_table(df):
    # Total missing values
    mis_val = df.isnull().sum()
    # Percentage of missing values
    mis_val_percent = 100 * df.isnull().sum() / len(df)
    # Make a table with the results
    mis_val_table = pd.concat([mis_val, mis_val_percent], axis=1)
    # Rename the columns
    mis_val_table_ren_columns = mis_val_table.rename(
        columns = {0 : 'Missing Values', 1 : '% of Total Values'})
    # Sort the table by percentage of missing descending
    mis_val_table_ren_columns = mis_val_table_ren_columns[
        mis_val_table_ren_columns.iloc[:,1] != 0].sort_values(
        '% of Total Values', ascending=False).round(1)
    # Print some summary information
    print ("Your selected dataframe has " + str(df.shape[1]) + "columns.\n"
          "There are " + str(mis_val_table_ren_columns.shape[0]) +
          " columns that have missing values.")
    # Return the dataframe with missing information
    return mis_val_table_ren_columns
```

```
missing_values_table(data)
```



Data Cleaning and Formatting

- Although we want to be careful to not discard information and should be careful when dropping columns, **if a column has a high percentage of missing values, then it probably will not be of much use.** For this project, we will remove any columns with more than 50% missing values. In general, be careful about dropping any information because even if it is not there for all the observations, it may still be useful for predicting the target value.

```
# Get the columns with > 50% missing
missing_df = missing_values_table(data);
missing_columns = list(missing_df[missing_df['% of Total Values'] > 50].index)
print('We will remove %d columns.' % len(missing_columns))
# Drop the columns
data = data.drop(columns = list(missing_columns))
```

- The rest of the missing values will have to be inputted (filled-in) using an appropriate strategy before doing machine learning.**



Recall the Problem Definition

Questions we are trying to answer:

- Does this dataset contain a set of independent variables that correlate to the energy star rating of the building?
- For buildings without energy star scores, can you infer what their score might be?



EDA

- **Univariate Analysis:**
 - Summarize single variables using ``describe()``
 - Possible Visualizations: Histogram (Distribution of values), Box plot (Identifying outliers), KDE plot (Kernel Density Estimation)
- **Bivariate & Multivariate Analysis:**
 - Analyse relationships between two or more variables
 - Possible Visualisations: Scatter plots (Numerical relationships), Pair plots (Multiple variable comparison), Heatmaps (Correlation analysis)



Univariate - Single Variable Plots

To begin the EDA, we will focus on a single variable, the **Energy Star Score**, because this is the target for our machine learning models. We can rename the column to `score` for simplicity and then start exploring this value.

```
# Rename the score
```

```
data = data.rename(columns = {'ENERGY STAR Score': 'score'})
```

```
# Histogram of the Energy Star Score
```

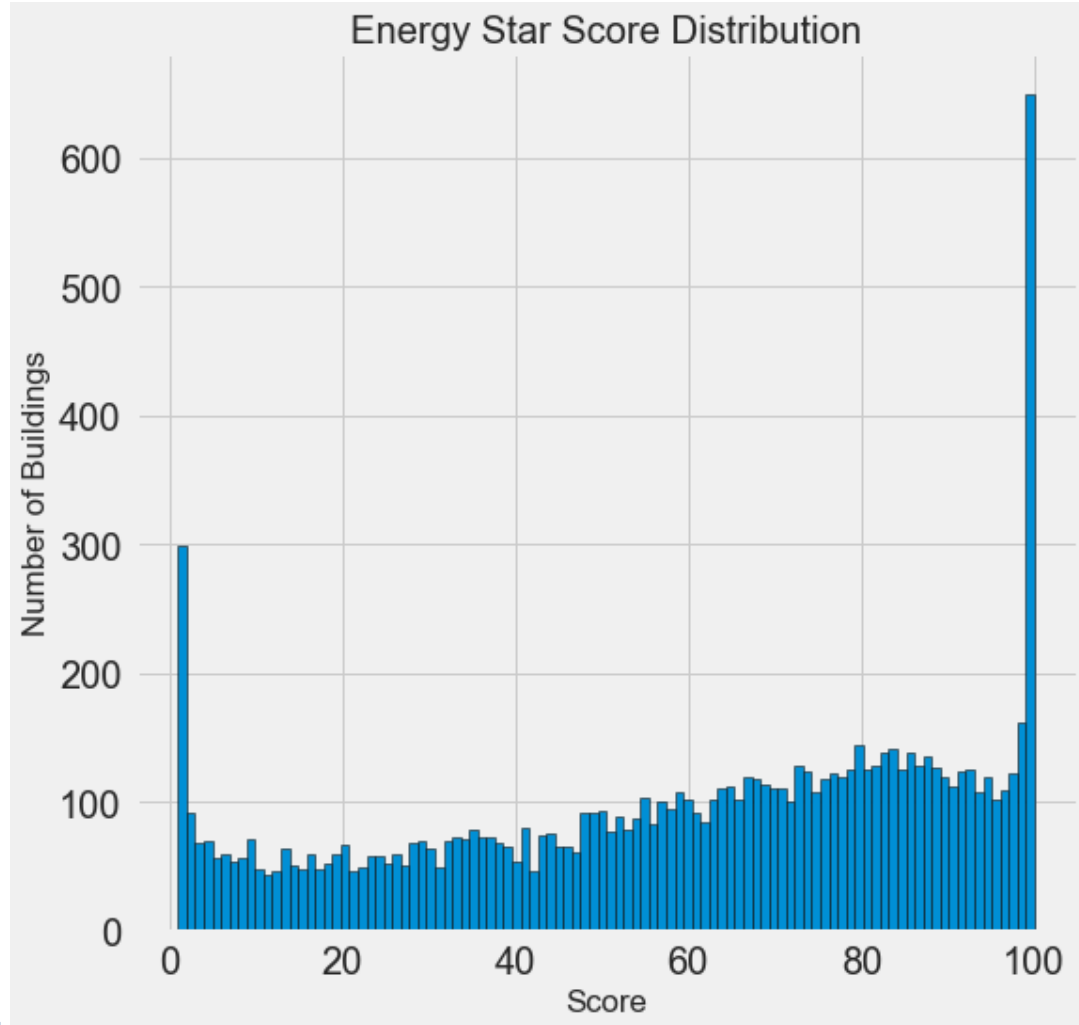
```
plt.hist(data['score'].dropna(), bins = 100, edgecolor = 'k');
```

```
plt.xlabel('Score'); plt.ylabel('Number of Buildings');
```

```
plt.title('Energy Star Score Distribution');
```



Univariate - Single Variable Plots



ENERGY STAR Score - 1-to-100 percentile ranking for specified building types, calculated in Portfolio Manager, based on self-reported energy usage for the reporting year.

Univariate - Single Variable Plots

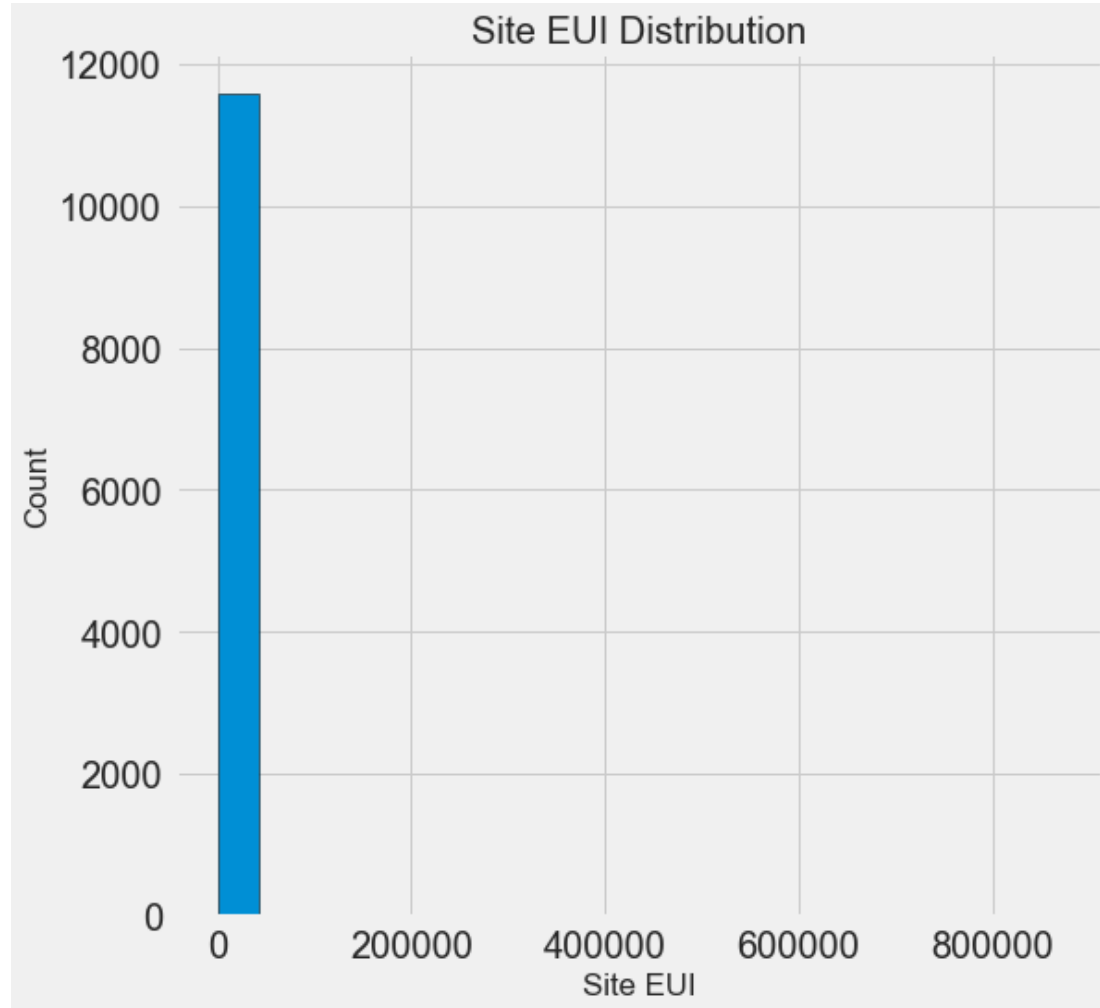
We can look at the **Energy Use Intensity (EUI)**, which is the total energy use divided by the square footage of the building. Here the energy usage is not self-reported, so this could be a more objective measure of the energy efficiency of a building. Moreover, this is not a percentile rank, so the absolute values are important, and we would expect them to be approximately normally distributed with perhaps a few outliers on the low or high end.

Histogram Plot of Site EUI

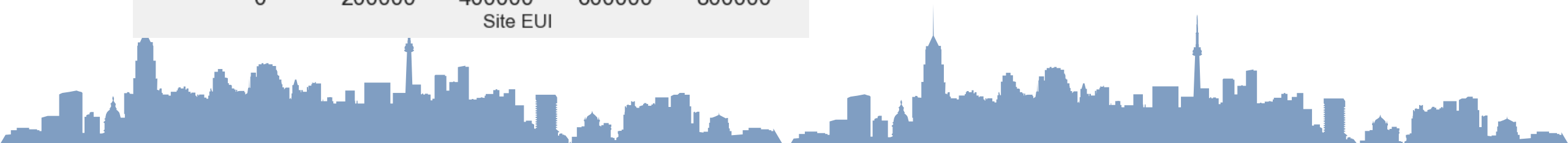
```
plt.hist(data['Site EUI (kBtu/ft²)'].dropna(), bins = 20, edgecolor = 'black');  
plt.xlabel('Site EUI');  
plt.ylabel('Count'); plt.title('Site EUI Distribution');
```



Univariate - Single Variable Plots



Outliers! The graph is incredibly skewed because of the presence of a few buildings with very high scores.



Univariate - Single Variable Plots

Let's look at the stats for this feature, the **Energy Use Intensity (EUI)**.

```
data['Site EUI (kBtu/ft²)'].describe()
```

```
count    11583.000000
mean      280.071484
std       8607.178877
min        0.000000
25%       61.800000
50%       78.500000
75%       97.600000
max      869265.000000
Name: Site EUI (kBtu/ft²), dtype: float64
```

```
data['Site EUI (kBtu/ft²)'].dropna().sort_values().tail(10)
```

```
3173    51328.8
3170    51831.2
3383    78360.1
8269    84969.6
3263    95560.2
8268   103562.7
8174   112173.6
3898   126307.4
7      143974.4
8068   869265.0
Name: Site EUI (kBtu/ft²), dtype: float64
```

Univariate - Single Variable Plots

One building is clearly far above the rest. Let's have a look at it.

```
data.loc[data['Site EUI (kBtu/ft²)'] == 869265, :]
```

Order	Property Id	Property Name	Parent Property Id	Parent Property Name	BBL - 10 digits	NYC Borough, Block and Lot (BBL) self-reported	NYC Building Identification Number (BIN)	Address 1 (self-reported)	Postal Code	Street Number	Street Name	Borough	DOF Gross Floor Area	Primary Property Type - Self Selected
8068	9984	4414323	234 Skillman	Not Applicable: Standalone Property	3028937502	3028937502	3390250 , 3338313	234 Skillman avenue	11211	234	SKILLMAN AVENUE	Brooklyn	61811.0	Multifamily Housing

Outliers can occur for many reasons: typos, malfunctions in measuring devices, incorrect units, or they can be legitimate but extreme values. Outliers can throw off a model because they are not indicative of the actual distribution of data



Removing Outliers

When removing outliers, we try to be as conservative as possible, using the definition of an extreme outlier:

- On the low end, an extreme outlier is below **First Quartile - 3*Interquartile Range**
- On the high end, an extreme outlier is above **Third Quartile + 3*Interquartile Range**

In this case, we will only remove the single outlying point and see how the distribution looks.

```
# Calculate first and third quartile
```

```
first_quartile = data['Site EU1 (kBtu/ft²)'].describe()['25%']
```

```
third_quartile = data['Site EU1 (kBtu/ft²)'].describe()['75%']
```

```
# Interquartile range
```

```
iqr = third_quartile - first_quartile
```

```
# Remove outliers
```

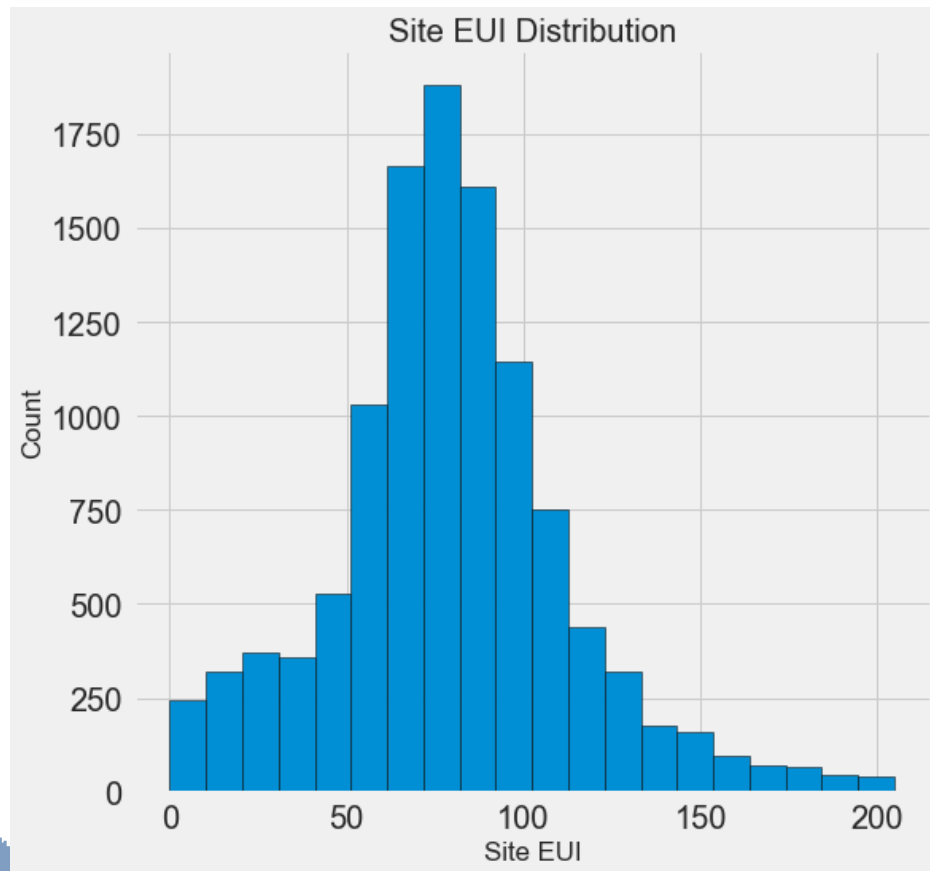
```
data = data[(data['Site EU1 (kBtu/ft²)'] > (first_quartile - 3 * iqr)) &  
            (data['Site EU1 (kBtu/ft²)'] < (third_quartile + 3 * iqr))]
```



Removing Outliers

Histogram Plot of Site EUI

```
plt.hist(data['Site EUI (kBtu/ft²)'].dropna(), bins = 20, edgecolor = 'black');  
plt.xlabel('Site EUI');  
plt.ylabel('Count'); plt.title('Site EUI Distribution');
```



After removing the outliers, we can get back to the analysis.

Correlations between Features and Target

- To quantify correlations between the features (variables) and the target, we can calculate the Pearson correlation coefficient.
- This is a measure of the strength and direction of a linear relationship between two variables: a value of -1 means the two variables are perfectly negatively linearly correlated and a value of +1 means the two variables are perfectly positively linearly correlated.

```
# Filter out non-numerical columns
```

```
numerical_data = data.select_dtypes(include=['number'])
```

```
# Find all correlations and sort
```

```
correlations_data = numerical_data.corr()['score'].sort_values()
```

```
# Print the most negative correlations
```

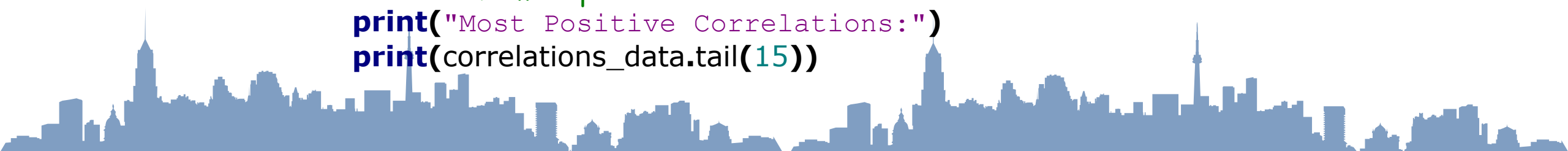
```
print("Most Negative Correlations:")
```

```
print(correlations_data.head(15), '\n')
```

```
# Print the most positive correlations
```

```
print("Most Positive Correlations:")
```

```
print(correlations_data.tail(15))
```



Correlations between Features and Target

Most Negative Correlations:

Site EUI (kBtu/ft ²)	-0.723864
Weather Normalized Site EUI (kBtu/ft ²)	-0.713993
Weather Normalized Source EUI (kBtu/ft ²)	-0.645542
Source EUI (kBtu/ft ²)	-0.641037
Weather Normalized Site Electricity Intensity (kWh/ft ²)	-0.358394
Weather Normalized Site Natural Gas Intensity (therms/ft ²)	-0.346046
Direct GHG Emissions (Metric Tons CO ₂ e)	-0.147792
Weather Normalized Site Natural Gas Use (therms)	-0.135211
Natural Gas Use (kBtu)	-0.133648
Year Built	-0.121249
Total GHG Emissions (Metric Tons CO ₂ e)	-0.113136
Electricity Use - Grid Purchase (kBtu)	-0.050639
Weather Normalized Site Electricity (kWh)	-0.048207
Latitude	-0.048196
Property Id	-0.046605

Name: score, dtype: float64

Most Positive Correlations:

Property Id	-0.046605
Indirect GHG Emissions (Metric Tons CO ₂ e)	-0.043982
Longitude	-0.037455
Occupancy	-0.033215
Number of Buildings - Self-reported	-0.022407
Water Use (All Water Sources) (kgal)	-0.013681
Water Intensity (All Water Sources) (gal/ft ²)	-0.012148
Census Tract	-0.002299
DOF Gross Floor Area	0.013001
Property GFA - Self-Reported (ft ²)	0.017360
Largest Property Use Type - Gross Floor Area (ft ²)	0.018330
Order	0.036827
Community Board	0.056612
Council District	0.061639
score	1.000000

Name: score, dtype: float64

- There are several strong negative correlations between the features and the target. The most negative correlations with the score are the different categories of Energy Use Intensity (EUI), Site EUI (kBtu/ft²) and Weather Normalized Site EUI (kBtu/ft²).
- The EUI is the amount of energy used by a building divided by the square footage of the buildings and is meant to be a measure of the efficiency of a building with a lower score being better. Intuitively, **these correlations then make sense: as the EUI increases, the Energy Star Score tends to decrease.**



Correlations between Features and Target

- To account for possible non-linear relationships, we can take square root and natural log transformations of the features and then calculate the correlation coefficients with the score.

```
# Select the numeric columns
numeric_subset = data.select_dtypes('number')
# Create columns with square root and log of numeric columns
for col in numeric_subset.columns:
    # Skip the Energy Star Score column
    if col == 'score':
        continue
    else:
        # Only apply square root to non-negative values
        numeric_subset['sqrt_' + col] = np.sqrt(numeric_subset[col].where(numeric_subset[col] >= 0))
        # Only apply log where values are greater than zero
        numeric_subset['log_' + col] = np.log(numeric_subset[col].where(numeric_subset[col] > 0))
# Select the categorical columns
categorical_subset = data[['Borough', 'Largest Property Use Type']]
# One hot encode
categorical_subset = pd.get_dummies(categorical_subset)
# Join the two dataframes using concat
# Make sure to use axis = 1 to perform a column bind
features = pd.concat([numeric_subset, categorical_subset], axis=1)
# Drop buildings without an energy star score
features = features.dropna(subset=['score'])
# Find correlations with the score
correlations = features.corr()['score'].dropna().sort_values()
# Display most negative correlations
correlations.head(15)
# Display most positive correlations
correlations.tail(15)
```



Correlations between Features and Target

Site EUI (kBtu/ft ²)	-0.723864
Weather Normalized Site EUI (kBtu/ft ²)	-0.713993
sqrt_Site EUI (kBtu/ft ²)	-0.699817
sqrt_Weather Normalized Site EUI (kBtu/ft ²)	-0.689019
sqrt_Weather Normalized Source EUI (kBtu/ft ²)	-0.671044
sqrt_Source EUI (kBtu/ft ²)	-0.669396
Weather Normalized Source EUI (kBtu/ft ²)	-0.645542
Source EUI (kBtu/ft ²)	-0.641037
log_Source EUI (kBtu/ft ²)	-0.622892
log_Weather Normalized Source EUI (kBtu/ft ²)	-0.620329
log_Site EUI (kBtu/ft ²)	-0.612039
log_Weather Normalized Site EUI (kBtu/ft ²)	-0.601332
log_Weather Normalized Site Electricity Intensity (kWh/ft ²)	-0.424246
sqrt_Weather Normalized Site Electricity Intensity (kWh/ft ²)	-0.406669
Weather Normalized Site Electricity Intensity (kWh/ft ²)	-0.358394

Name: score, dtype: float64

sqrt_Order	0.028662
Borough_Queens	0.029545
Largest Property Use Type_Supermarket/Grocery Store	0.030038
Largest Property Use Type_Residence Hall/Dormitory	0.035407
Order	0.036827
Largest Property Use Type_Hospital (General Medical & Surgical)	0.048410
Borough_Brooklyn	0.050486
log_Community Board	0.055495
Community Board	0.056612
sqrt_Community Board	0.058029
sqrt_Council District	0.060623
log_Council District	0.061101
Council District	0.061639
Largest Property Use Type_Office	0.158484
score	1.000000

Name: score, dtype: float64

- The log and square root transformations do not seem to have resulted in any stronger relationships.
- There are no strong positive linear relationships although we do see that a building type of office (Largest Property Use Type_Office) is slightly positively correlated with the score.



Two-Variable Plots

With the code below we will plot two numeric variables against one another and use colour to represent a third categorical variable.

```
# Extract the building types
```

```
features['Largest Property Use Type'] = data.dropna(subset = ['score'])['Largest Property Use Type']
```

```
# Create a list of buildings with more than 100 measurements
```

```
types = data.dropna(subset=['score'])
```

```
types = types['Largest Property Use Type'].value_counts()
```

```
types = list(types[types.values > 100].index)
```

```
# Limit to building types with more than 100 observations (from previous code)
```

```
features = features[features['Largest Property Use Type'].isin(types)]
```

```
# Use seaborn to plot a scatterplot of Score vs Log Source EUI
```

```
sns.lmplot(
```

```
    x='Site EUI (kBtu/ft²)',
```

```
    y='score',
```

```
    hue='Largest Property Use Type',
```

```
    data=features,
```

```
    scatter_kws={'alpha': 0.8, 's': 60},
```

```
    fit_reg=False,
```

```
    height=12, # Use 'height' instead of 'size'
```

```
    aspect=1.2
```

```
)
```

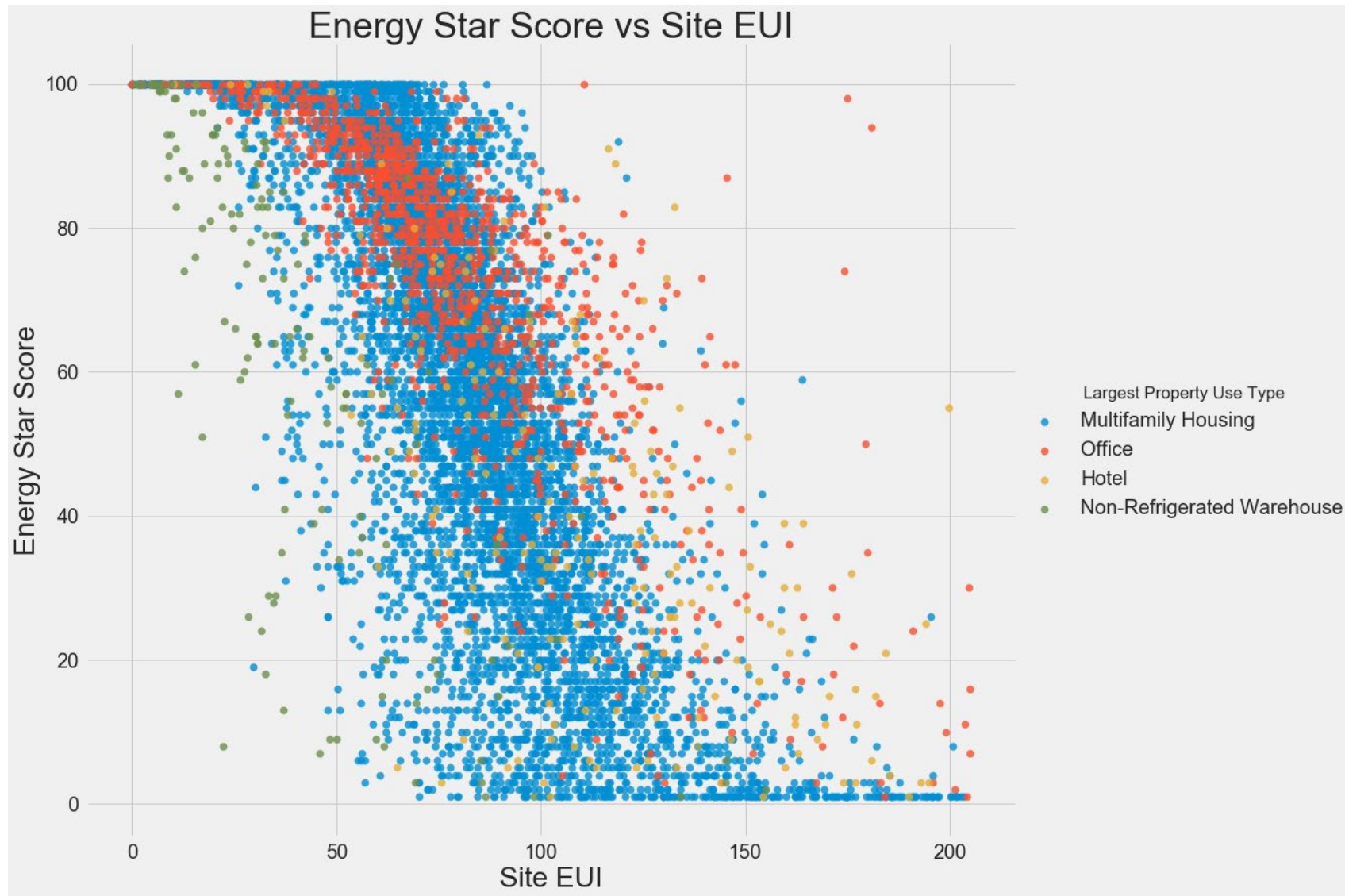
```
# Plot labeling
```

```
plt.xlabel("Site EUI", size=28)
```

```
plt.ylabel("Energy Star Score", size=28)
```

```
plt.title("Energy Star Score vs Site EUI", size=36)
```

Two-Variable Plots



Feature Engineering and Selection

If we don't feed the model the correct data, then we are setting it up to fail and we should not expect it to learn!

In this project, we will take the following steps for feature engineering:

- Select only the numerical variables and two categorical variables (borough and property use type)
- Add in the log transformation of the numerical variables
- One-hot encode the categorical variables

For feature selection, we will do the following:

- Remove collinear features

We will discuss collinearity (also called multicollinearity) when we get to that process!



The following code:

- Selects the numeric features and adds in log transformations of all the **numeric features**: compresses large values and stretches out small ones, turning exponential-looking distributions into something more manageable.
- Selects and one-hot encodes the categorical features: Transforming **categorical variables** into numeric form (multiple binary columns).
- Joins the sets of features together.

```
# Copy the original data
features = data.copy()
# Select the numeric columns
numeric_subset = data.select_dtypes('number')
# Create columns with log of numeric columns
for col in numeric_subset.columns:
    # Skip the Energy Star Score column
    if col == 'score':
        next
    else:
        numeric_subset['log_' + col] = np.log(numeric_subset[col])

# Select the categorical columns
categorical_subset = data[['Borough', 'Largest Property Use Type']]

# One hot encode
categorical_subset = pd.get_dummies(categorical_subset)

# Join the two dataframes using concat
# Make sure to use axis = 1 to perform a column bind
features = pd.concat([numeric_subset, categorical_subset], axis = 1)
features.shape
```



Remove Collinear Features

- While variables in a dataset are usually correlated to a small degree, highly collinear variables can be redundant in the sense that we only need to retain one of the features to give our model the necessary information.
- Removing collinear features is a method to reduce model complexity by decreasing the number of features and can help to increase model generalization. It can also help us to interpret the model because we only have to worry about a single variable, such as EUI, rather than how both EUI and weather normalized EUI affect the score.
- There are a number of methods for removing collinear features, such as using the [Variance Inflation Factor](#). We will use a simpler metric and remove features that have a correlation coefficient above a certain threshold with each other (not with the score because we want variables that are highly correlated with the score!)



Remove Colinear Features

At this point, we have 11319 observations (buildings) with 109 different features (one column is the score). Not all of these features are likely to be important for predicting the score, and several of these features are also redundant because they are highly correlated.

Highly collinear features have a significant correlation coefficient between them. For example, in our dataset, the Site EUI and Weather Norm EUI are highly correlated because they are just slightly different means of calculating the energy use intensity.

```
# Drop NaN values for the selected columns
plot_data = data[['Weather Normalized Site EUI (kBtu/ft²)', 'Site EUI (kBtu/ft²)']].dropna()
# Compute the correlation coefficient
correlation = np.corrcoef(plot_data['Weather Normalized Site EUI (kBtu/ft²)'],
                          plot_data['Site EUI (kBtu/ft²)'])[0, 1]
# Scatter plot
plt.plot(plot_data['Site EUI (kBtu/ft²)'], plot_data['Weather Normalized Site EUI (kBtu/ft²)'], 'bo')
# Labels and title
plt.xlabel('Site EUI')
plt.ylabel('Weather Norm EUI')
plt.title(f'Weather Norm EUI vs Site EUI, R = {correlation:.4f}') # Using f-string for better readability
# Show the plot
plt.show()
```



Remove Collinear Features

```
def remove_collinear_features(x, threshold):
    # Dont want to remove correlations between Energy Star Score
    y = x['score']
    x = x.drop(columns = ['score'])
    # Calculate the correlation matrix
    corr_matrix = x.corr()
    iters = range(len(corr_matrix.columns) - 1)
    drop_cols = []
    # Iterate through the correlation matrix and compare correlations
    for i in iters:
        for j in range(i):
            item = corr_matrix.iloc[j:(j+1), (i+1):(i+2)]
            col = item.columns
            row = item.index
            val = abs(item.values)
            # If correlation exceeds the threshold
            if val >= threshold:
                # Print the correlated features and the correlation value
                # print(col.values[0], "|", row.values[0], "|", round(val[0][0], 2))
                drop_cols.append(col.values[0])
    # Drop one of each pair of correlated columns
    drops = set(drop_cols)
    x = x.drop(columns = drops)
    x = x.drop(columns = ['Weather Normalized Site EUI (kBtu/ft²)',
                          'Water Use (All Water Sources) (kgal)',
                          'log_Water Use (All Water Sources) (kgal)',
                          'Largest Property Use Type - Gross Floor Area (ft²)'])
    # Add the score back in to the data
    x['score'] = y
    return x
```

The following code removes the collinear features based on a threshold we select for the correlation coefficients by removing one of the two features that are compared. It also prints the correlations that it removes so we can see the effect of adjusting the threshold. We will use a threshold of 0.6 which removes one of a pair of features if the correlation coefficient between the features exceeds this value.



Remove Collinear Features

```
# Remove the collinear features above a specified correlation  
coefficient  
features = remove_collinear_features(features, 0.6);  
  
# Remove any columns with all na values  
features = features.dropna(axis=1, how = 'all')  
features.shape
```

You should get the output (11319, 65).

Our final dataset now has 64 features (one of the columns is the target). This is still quite a few, but mostly it is because we have one-hot encoded the categorical variables.

There are other feature selection steps to take, but for now we will keep all the features we have and see how the model performs.



Save the results for Future Analysis

```
no_score = features[features['score'].isna()]
score = features[features['score'].notnull()]
print(no_score.shape)
print(score.shape)
```

Output:

```
(1858, 65)
(9461, 65)
```

Separate out the features and targets

```
features = score.drop(columns='score')
```

```
targets = pd.DataFrame(score['score'])
```

Replace the inf and -inf with nan

```
features = features.replace({np.inf: np.nan, -np.inf: np.nan})
```

Split into 70% training and 30% testing set

```
from sklearn.model_selection import train_test_split
```

```
X, X_test, y, y_test = train_test_split(features, targets, test_size = 0.3, random_state = 42)
```



Save the results for Future Analysis

```
print(X.shape)
print(X_test.shape)
print(y.shape)
print(y_test.shape)

# Save the no scores data
no_score.to_csv('no_score.csv', index = False)

# Save the scores training and testing data
X.to_csv('training_features.csv', index = False)
X_test.to_csv('testing_features.csv', index = False)
y.to_csv('training_labels.csv', index = False)
y_test.to_csv('testing_labels.csv', index = False)
```

Output:

```
(6622, 64)
(2839, 64)
(6622, 1)
(2839, 1)
```

